

CONV 高性能计算 优化总结

提交前精简版

正式成绩	50651.6539 GFLOPS
理论上限占比	70.35%
正确性	全部 PASS · 最大误差 0

成功、失败、原因、当前瓶颈与提交说明

更新至 2026-09-01 CST

目标：在不修改 `bench_conv.c`、不改变 `conv2d` 接口和计算语义的前提下，用单 NUMA、最多 38 核优化二维 valid convolution。

当前提交实现：`conv/conv2d.c`

最后更新：2026-09-01 CST

1. 一页结论

当前实现已经从普通 C 四重循环，优化为 38 核 OpenMP + 512 位 SVE/SME + 16×64 外积微内核。

最新官方提交成绩如下：

Case	GFLOPS	单项上限占比
1	12086.9473	67.15%
2	11581.6171	64.34%
3	13337.1850	74.10%
4	13645.9045	75.81%
总计	50651.6539	70.35% / 72000

当前排名为第 11 名。相较上一版正式成绩 49667.4161 GFLOPS，提升 984.2378 GFLOPS，约 1.98%。

服务器普通 DDR 多轮测试的总分中位数约为 51296.8992 GFLOPS，正式成绩比该中位数低约 1.26%，处于共享服务器实测波动区间内。

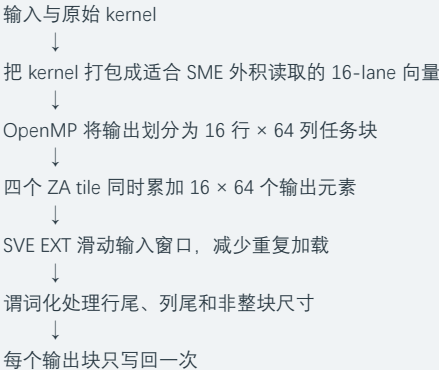
从初版到当前正式成绩：

阶段	四项总性能	相对初版
原始四重循环	254.3092	1×
显式 SVE	5833.8272	22.94×
当前 SME 正式成绩	50651.6539	199.17×

最重要的结论是：SME 外积映射是数量级提升的来源；普通预取、参数微调和片上内存无法再带来同量级收益。

2. 当前代码做了什么

当前 `conv2d.c` 的主路径可以概括为：



2.1 SME 16×64 外积

SME 的 `FMOPA` 一次完成一个 16×16 FP32 外积。代码同时使用四个 ZA tile：

ZA0: 输出列 0-15
ZA1: 输出列 16-31

ZA2: 输出列 32-47
ZA3: 输出列 48-63

kernel 的 16 个系数对应 16 个输出行，input 向量对应 16 个输出列。一条外积指令能够更新 256 个累加位置，这是相比普通 SVE 获得数量级提升的核心。

2.2 Kernel 打包

原始 kernel 按“物理输入行、kernel 列、16 个输出行系数”打包。每组正好为 16 个 float，即 64 字节：

- 与缓存行对齐；
- SME 可以连续加载一整个系数向量；
- 无效的顶部和底部 lane 填 0；
- 保持原始 kernel row -> kernel column 累加顺序。

2.3 输入滑动窗口

连续 kernel 列对应高度重叠的 input 窗口。代码用五个 SVE 向量维护 64 列输出所需的窗口，通过 EXT 每次向右滑动一列，只补充新进入的数据，避免为每个 kernel 列重复加载四个完整输入向量。

2.4 通用任务分派

实现没有判断 Case 编号或完整公开尺寸，而是依据通用工作量选择策略：

- 中小 kernel：细粒度 static 任务，减少线程收尾不均衡；
- 宽 kernel：连续列分区配合 guided 调度，兼顾负载均衡和缓存局部性；
- kernel 宽度、输出块数量和尾部长度决定预取与尾部路径；
- 不满足 SME 运行条件时回退到 SVE 或普通 C 路径。

3. 成功的优化

优化	得到的结果	成功原因	是否保留
显式 SVE 多累加器	从初版约 254 提升到约 5834 总分	消除标量依赖链，一次计算多个输出列	保留为回退路径
SME 单 ZA 16×16	Case 1 达到约 5.2k	把二维卷积复用转换为矩阵外积	是
四 ZA 16×64	Case 1 提升到约 9.3k	同一 kernel 向量复用到四组输出列	是
SVE 滑动与跨组窗口携带	大 kernel 获得明显收益	减少重叠 input 窗口加载和寄存器搬运	是
64 字节对齐 kernel 打包	多轮总分约提升 2%-3%	每个 16-lane 系数向量与缓存行对齐	是
Fine-static / coarse-guided 调度	Case 1/2 和 Case 4 获得稳定改善	减少线程长尾，同时避免任务过碎	是
中小 kernel carry-tail	Case 1 多轮约提升 4%，Case 2 小幅提升	尾部继续复用已有窗口，只补一个新 lane	是
BiSheng -O3 -ffast-math 内核编译	热循环机器码更紧凑	测试程序与内核分开编译，正确性仍为 0 误差	是

这些成功优化共同形成当前正式版本，而不是依赖某一个公开 Case 的完整尺寸。

4. 失败的优化及原因

失败实验同样重要，因为它们说明剩余瓶颈不在哪里。

尝试	实测结果	失败原因	最终处理
继续增加普通 SVE 累加器和展开	只能小幅变化，远低于 SME	没有改变“一条 FMA 只更新一个向量”的结构	停止投入
精确去掉 SME 无效行 lane	回退约 2%-5%	谓词与边界控制成本高于减少的无效计算	删除
SVE2 TBL / SPLICE 构造窗口	接近持平或回退	指令更少但延迟、寄存器压力和后端停顿更高	删除
每列直接加载四个 input 向量	固定核回退约 10.9%	LD1W 和寄存器搬运数量显著增加	删除
kernel-height 分带 ZA 行环	Case 1 最好仅约 8.4k	虽减少 8%-14% 外积，但部分和反复写回/重载更昂贵	删除
packed-kernel 缓存分段	回退约 6%-10%	ZA spill/reload 流量超过 L1 复用收益	删除
尾部 2/4 路展开	Case 1 持续回退	活跃向量过多，增加寄存器和 load 队列压力	删除
KBLAS / KuDNN / im2col GEMM	慢或无法满足精度	数据展开、通用库开销和累加顺序不适合本题	删除
FFT / Winograd / 低秩分解	不适合作为通用实现	kernel 为随机大核，且严格误差阈值限制变换顺序	不采用
宽 kernel carry-tail（65-96）	81、83 宽三轮均回退	宽核下直接尾部加载更适合当前寄存器与缓存状态	删除

4.1 片上内存为何不采用

服务器 Flat 模式把约 4 GB 的 memory-only 节点暴露为独立 NUMA。单轮曾得到 51917.3699 GFLOPS，但多轮复测否定了其稳定性。

节点 7 的五轮公开 Case 配对测试：

指标	普通 DDR	配对片上内存
总分轮次中位数	51296.8992	51022.7257
配对差值中位数	-	-161.0283，约 -0.31%

相邻非公开尺寸约提升 1.08%，但不同 Case 有升有降；节点 5、13、15 的方向也互相矛盾。说明此前 Case 4 的 14.5k 是共享环境下的单次高点，不是可泛化收益。

此外，numactl -N 7 -m 23 同时涉及计算 NUMA 和 memory-only NUMA 两个 ID，合规关系也未得到官方书面确认。因此当前 run.sh 不使用片上内存。

5. 当前瓶颈

5.1 KH+15 额外物理行

当前内核一次计算 16 个输出行，需要扫描 kernelHeight + 15 个物理输入行。评分只计算真实的 kernelHeight 行，因此小 kernel 的结构损失更明显：

Case	KH	有效计算比例 $\frac{KH}{KH+15}$	当前映射理想上限
1	39	72.22%	14051.56
2	41	73.21%	14244.57
3	55	78.57%	15286.86
4	81	84.38%	16416.00

这解释了为什么排行榜成熟方案通常随 kernel 变大而提高 GFLOPS，也解释了为什么 Case 1/2 不能只靠预取参数达到 14.5k。

5.2 SME 热循环供给

热循环仍需要同时完成：

- kernel 向量加载；
- input 向量加载与 EXT 滑动；
- 四条 FMOPA；
- 地址计算和 OpenMP 任务推进；
- ZA 最终写回。

Case 1/3 已较接近当前映射的实现效率上限；Case 2/4 的后端停顿、缓存/TLB和线程长尾更加明显。

6. 正确性与规则检查

当前版本满足以下提交条件：

- 未修改 conv/bench_conv.c；
- 未修改 conv2d 函数接口；
- 四个官方 Case 全部 PASS，最大误差为 0；
- 多组额外尺寸、尾部尺寸和边界尺寸 PASS；
- 最多使用 38 个 CPU 核心；
- 当前提交脚本只绑定一个普通 DDR NUMA；
- 没有按完整公开输入尺寸硬编码；
- 没有结果缓存、漏算、伪造计时或修改测试流程；
- conv_init/ 初版备份未修改。

7. 提交版本说明

提交目录为 conv/，其中：

文件	用途
conv2d.c	当前优化实现
bench_conv.c	官方测试程序，不修改
run.sh	加载 BiSheng、编译并运行四个官方 Case
README.md	官方赛题说明

当前 run.sh：

- 默认 OMP_NUM_THREADS=38；
- 默认普通 DDR NUMA 7；
- 使用 BiSheng 5.0.0.2；
- benchmark 使用严格 -O3；
- conv2d.c 单独使用 -O3 -ffast-math -funroll-loops；
- 不使用片上内存候选方案。

正式提交前应重新压缩当前 conv/，不要直接复用旧 conv.zip，并确认压缩包根目录能直接看到 run.sh。

8. 后续优化方向

当前 70.35% 距离 80% 总分线还差 6948.3461 GFLOPS，距离 90% 还差 14148.3461 GFLOPS。后续优先级如下：

1. 完整 SME 稳态流水：手写或内联汇编实现跨两个 kernel shift 的 load/EXT/FMOPA 双缓冲；固定核微基准先达到至少 10% 收益，再进入全 Case。
2. 输出写回重叠：在不产生部分和、不增加输出流量的前提下，将 ZA drain 与下一 tile 准备重叠。
3. 通用多微内核分派：依据 kernel 宽高、尾长、packed-kernel 大小和输出任务数选择实现；必须在相邻非公开尺寸上同步获益。
4. 突破 KH+15 的新映射：必须保持输出只写一次和原始 FP32 累加顺序；任何需要反复保存完整部分和的方案不再实现。
5. 稳定性优化：继续记录多轮中位数和最差轮，避免把共享服务器单次峰值当成代码收益。

70% 只是当前阶段结果，不是终点；但 80%-90% 必须依靠新的微内核流水或数学映射，而不是继续扫描小参数。

9. 最终结论

- 成功主线：OpenMP + SVE/SME 16×64 外积 + 对齐打包 + 滑动窗口 + 通用任务分派。
- 当前正式成绩：50651.6539 GFLOPS，理论上限的 70.35%。
- 当前可信多轮水平：普通 DDR 总分中位数约 51296.8992 GFLOPS。
- 已否决方向：片上内存、分带部分和、TBL 窗口、直接全加载、缓存分段、通用库替换和宽核 carry-tail。
- 提交选择：保留当前普通 DDR 版本，不加入未经证明或存在合规疑问的实验路径。

更细的实验数据、参数扫描和开发计划仍保留在：

- [docs/optimization-report.md](#)
- [docs/2026-08-28-Conv下一阶段高性能优化开发计划.md](#)